

ELE 216 Laboratoire 2 : Révision langage C

Écrivez un programme complet avec Makefile pour chacun des problèmes suivants. Vous devez réaliser ce laboratoire seul, pendant la période. Il vous est permis de discuter avec vos collègues, consulter les chargés de laboratoires et l'internet.

Il est permis de réutiliser certaines fonctions réalisées dans les numéros précédents pour les numéros suivants.

Vous devez faire valider votre travail avant de quitter. On vous demandera d'expliquer clairement le fonctionnement de certains éléments de votre implémentation.

Problème 1 : Tableaux 1D

Écrire trois fonctions :

`int somme_tableau(int tab[], int n)` retournant la somme des éléments.

`int indice_max(int tab[], int n)` retournant l'indice de la valeur maximale.

`void afficher_tableau(int tab[], int n)` qui affiche les éléments d'un tableau sur une seule ligne, séparés par des espaces (pas d'espace après le dernier élément).

Écrire un `main` de test qui affiche le contenu, la somme et la valeur maximale d'un tableau de 10 entiers initialisé dans le code.

Problème 2 : Tableaux 2D

Écrire trois fonctions :

`void afficher_matrice(int mat[][5], int lignes)` qui affiche une matrice 2D sous forme de grille (lignes et colonnes alignées).

`void sommes_lignes(int mat[][5], int lignes, int resultats[])` qui stocke dans `resultats[i]` la somme de la ligne `i`.

`void sommes_colonnes(int mat[][5], int lignes, int resultats[])` qui fait de même pour les colonnes.

Écrire un `main` qui affiche la matrice ainsi que les sommes pour une matrice 4×5 initialisée dans le code.

Problème 3 : Structures

NOTE : À partir de ce numéro, les paramètres des fonctions ne sont plus imposés. C'est à vous de déterminer quels paramètres sont nécessaires.

Définissez une structure `capteur_t` qui modélise un capteur IoT. Chaque `capteur_t` contient un identificateur (`int`), un titre (`char[20]`, exemple "temperature"), la dernière valeur

mesurée (float), une unité de mesure (char[8], exemple "N/m" ou "mV" ou "kWh") et un état booléen d'activation (int).

Écrivez 3 fonctions :

`void capteur_afficher()` qui affiche tous les champs du capteur sur le terminal dans un format utile au débogage.

`void capteur_init()` qui reçoit un pointeur vers un `capteur_t` ainsi que toutes les valeurs nécessaires à son initialisation. Attention de bien faire une copie profonde des chaînes de caractères.

`void capteur_mise_a_jour()` qui reçoit un pointeur vers un capteur ainsi que la nouvelle valeur mesurée.

Écrire un `main` qui crée 2 capteurs, les initialise et les affiche. Il met ensuite à jour la valeur de l'un des deux et l'affiche à nouveau.

Problème 4 : Tableaux dynamiques : allocation et libération

Écrire une fonction `int *creer_tableau()` qui alloue dynamiquement un tableau de `n` entiers avec `malloc`, initialise chaque élément à `i * 2` (où `i` est l'indice), et retourne le pointeur.

Écrire un `main` qui crée le tableau, l'affiche, puis libère la mémoire correctement.

Problème 5 : Tableaux 2D dynamiques : allocation ligne par ligne

NOTE : une "matrice" est terme mathématique général qui correspond à un tableau 2D en C.

Écrire une fonction `int **creer_matrice(int lignes, int colones)` qui alloue dynamiquement une matrice d'entiers (tableau de tableaux, chaque ligne est allouée séparément).

Venez voir le chargé de lab si vous avez de la difficulté à visualiser la structure. C'est important!

Écrire une fonction `void liberer_matrice(int **matrice, int lignes)` qui libère la mémoire correctement.

Le `main` alloue une matrice 5 par 8, la remplit avec des valeurs aléatoires entre 20 et 80, l'affiche et la libère.

Problème 6 : Tableau de structures

Définir une structure `point2D_t` avec deux champs `float x` et `float y`.

Écrire une fonction `float distance(point2D_t *a, point2D_t *b)` qui calcule la distance entre deux points.

Écrire un `main` qui déclare un tableau statique de 5 `point2D_t` puis affiche toutes les distances entre points consécutifs (1 vers 2, 2 vers 3, etc.) ainsi que la distance totale.

Mini app : Station météo

Une station météo gère un réseau de capteurs IoT. Elle enregistre les mesures dans un historique et calcule des statistiques sur les lectures.

Vous devez réutiliser toutes les fonctions pertinentes des numéros précédents.

Structures à définir

Réutiliser `capteur_t` du problème 3.

Définir une structure `station_t` contenant :

1. un tableau dynamique de `capteur_t`
2. le nombre de capteurs dans le tableau
3. le nombre de mesures effectuées
4. une matrice dynamique de `float`. lignes = capteurs, colonnes = mesures.

Fonctions à écrire

```
station_t station_creer(int nb_capteurs, int nb_mesures)
```

Alloue dynamiquement le tableau de capteurs et la matrice d'historique. Initialise les capteurs avec des valeurs prédéfinies (au moins 3 capteurs : température, pression, humidité).

```
void station_enregistrer_mesure(station_t *s, int idx_mesure, float valeurs[]);
```

Stocke dans la colonne `idx_mesure` de la matrice les nouvelles valeurs mesurées, et met à jour la valeur courante de chaque capteur.

```
float station_moyenne_capteur(station_t *s, int idx_capteur);
```

Retourne la moyenne de toutes les mesures d'un capteur donné.

```
int station_indice_valeur_max(station_t *s, int idx_capteur);
```

Retourne l'indice de la mesure maximale pour un capteur donné.

```
void station_afficher_historique(station_t *s);
```

Affiche la matrice d'historique sous forme de grille, capteurs en lignes, relevés en colonnes.

```
void station_afficher_stats(station_t *s);
```

Pour chaque capteur, affiche son nom, sa moyenne, ainsi que la valeur et l'indice de sa mesure maximale.

```
void station_liberer(station_t *s);
```

Libère le tableau de capteurs et la matrice d'historique.

Programme principal (main)

Le main orchestre la simulation :

1. Crée une station avec 3 capteurs et 5 relevés
2. Enregistre 5 séries de valeurs simulées (aléatoires, avec des valeurs réalistes)
3. Affiche l'historique complet sous forme de grille
4. Affiche les statistiques par capteur
5. Libère toute la mémoire